



FormFlux

A self-hostable, production-ready alternative to FormSubmit.co. Email delivery, spam protection, file uploads, webhooks and an owner dashboard for any plain HTML form.

What this document covers

The complete architecture of the FormFlux engine, the request-by-request workflow of a form submission, every special field and how it changes behaviour, the data model, the owner dashboard, the plan gating model, the full build log of what was created across five phases, deployment steps, and a workflow flowchart on the final page.

API RUNTIME	Cloudflare Workers + Hono (TypeScript)
DATABASE	Supabase Postgres with Row Level Security
AUTH	Supabase Auth (magic link)
EMAIL	Resend
FILE STORAGE	Cloudflare R2
RATE LIMITING	Cloudflare KV
RETRIES	Cloudflare Queues
DASHBOARD	Next.js 14 App Router + Tailwind CSS

Executive summary

FormFlux replaces the small backend everyone writes for contact forms. A developer points a plain HTML form at a FormFlux URL and immediately gets email delivery, spam filtering, validation, file attachments, webhook fan-out, an autoreply, and a dashboard to browse everything — without writing or hosting server code.

It is a feature-complete replica of FormSubmit.co: every documented special field behaves identically, so snippets copied from FormSubmit's documentation work unchanged. On top of that parity, FormFlux adds an owner dashboard, real analytics, per-form configuration, and an unlimited submission archive.

Design goals

- Zero-config first run: a developer should get a working form before creating an account.
- Nothing paid required: the entire stack runs on free tiers (Cloudflare, Supabase, Resend).
- Never lose a submission: spam is flagged and stored, never silently discarded, so false positives can be recovered.
- No secrets in page source: UUID endpoints mean the delivery address never appears in HTML.
- Owner control: retention windows, rate limits, validation rules, and blocklists are all per-form settings.

What exists today

Directory	Responsibility
<code>worker/</code>	The API. Hono app on Cloudflare Workers handling submissions, verification, file downloads, webhooks, retries, the daily retention cron, and authenticated dashboard endpoints.
<code>dashboard/</code>	Next.js 14 app serving both the public marketing site (landing, docs, pricing) and the authenticated owner dashboard (forms, settings, submissions, webhooks, analytics, archive).
<code>supabase/</code>	schema.sql for a fresh install plus migrations/002_dashboard.sql for existing installs. Four tables, all under Row Level Security.
<code>docs/</code>	This document and the pdfkit script that generates it.

One-line summary

FormFlux is a Cloudflare Worker that turns an HTTP form POST into an email, a database row, a set of webhook deliveries, and an autoreply — plus a Next.js dashboard that lets the form's owner configure and inspect all of it.

SECTION 02

System architecture

FormFlux is two deployable units that share one Supabase project. They never share a secret: the Worker holds the service-role key, the dashboard only ever uses the anon key and relies on Row Level Security.

The two units

WORKER (THE ENGINE)

- Runs at the edge on Cloudflare Workers, so submissions are handled close to the sender with no cold-start server to keep warm.
- Holds SUPABASE_SERVICE_KEY, RESEND_API_KEY, and HCAPTCHA_SECRET as Worker secrets. Because service-role queries bypass RLS, every authenticated route re-checks ownership explicitly.
- Bindings: RATE_LIMIT_KV (rate limit counters), FORM_UPLOADS (R2 bucket for attachments), WEBHOOK_QUEUE (retry queue), and a cron trigger at 03:00 UTC for the retention sweep.

DASHBOARD (THE CONTROL SURFACE)

- Next.js 14 App Router. Server Components read Supabase through the request's cookies, so every query is filtered by RLS to the signed-in owner's rows.
- Server Actions handle writes (create form, save settings, delete, toggle spam flag). Client components handle interactive tables, charts, and forms.
- Two privileged operations — resending a notification email and sending a test webhook — are delegated to the Worker with the user's Supabase access token, because they need the Resend key.

Trust boundaries

Boundary	How it is controlled
Browser !' Worker	Public. Anyone may POST a submission. Protection comes from the honeypot, captcha, rate limits, blocklists, and validation rules — not from authentication.
Browser !' Supabase	Anon key only. Row Level Security restricts every read and write to rows owned by the authenticated user.
Dashboard !' Worker	Supabase access token in an Authorization: Bearer header. The Worker verifies the token, then checks that the user owns the target form.
Worker !' Supabase	Service-role key. Full access, so ownership checks are performed in application code.

SECTION 03

Data model

Four tables, all with Row Level Security enabled. A trigger on auth.users creates a free profiles row for every new signup, so the dashboard never has to handle a missing profile.

profiles

One row per authenticated user. Holds the plan flag that drives all feature gating.

```
id          uuid primary key, references auth.users
email       text
plan        text 'free' | 'pro' (default 'free')
created_at  timestampz
```

forms

One row per endpoint. Every dashboard setting is a column here, which is why the owner never has to hand-edit hidden inputs unless they want a one-off override.

```
id          uuid primary key          -- this is the /f/:formId in the endpoint
owner_id    uuid -> profiles(id)      -- null for unclaimed zero-config forms
name        text
target_email text
is_verified boolean                  -- gates delivery
verification_token text
redirect_url text                    -- default redirect, overridden by _next
default_subject text                 -- overridden by _subject
default_cc   text                   -- overridden by _cc
webhook_urls text[]                  -- generic JSON destinations
slack_webhook_url text
discord_webhook_url text
captcha_enabled boolean              -- overridden by _captcha=false
blacklist_phrases text[]             -- merged with per-submission _blacklist
autoreply_enabled boolean
autoreply_template text              -- overridden by _autoresponse
validation_rules jsonb               -- { field: { required, regex } }
retention_days int                   -- 0 keeps forever
rate_limit_per_hour int              -- default 100
```

submissions

One row per accepted submission, including spam. The meta column is the audit trail the dashboard drawer reads to show which special fields were used and whether captcha ran.

```

id          uuid  primary key
form_id     uuid  -> forms(id) on delete cascade
data        jsonb  -- submitted fields, special fields stripped
files       text[] -- R2 object keys
ip_address  text
is_spam     boolean -- true = stored but not emailed or webhooked
delivered   boolean -- owner notification actually sent
meta        jsonb  -- specialFields[], captcha, spamReasons[], redirect, plan
search_text text  GENERATED ALWAYS AS (data::text) STORED
created_at  timestampz

```

Why search_text exists

PostgreSQL cannot ILIKE across an entire jsonb document, and enumerating field names would miss custom fields. A stored generated column holding the payload as text lets one ILIKE search every field of every submission, with a pg_trgm GIN index to keep it fast.

webhook_logs

One row per delivery attempt, including retries and manual tests. form_id is present so test deliveries (which have no submission) are still visible and still covered by RLS.

```

id          uuid  primary key
submission_id uuid  -> submissions(id) -- null for test deliveries
form_id     uuid  -> forms(id)
destination text
status_code int      -- 0 when the request never completed
attempt     int      -- 1..5
response_snippet text   -- first 300 chars of the response
source      text     -- generic | slack | discord | formsubmit |
test
created_at  timestampz

```

Row Level Security summary

Table	Policy
profiles	Select, insert, and update only where auth.uid() = id.
forms	All operations where auth.uid() = owner_id.
submissions	Select, update, and delete where the parent form is owned by auth.uid(). Inserts come from the Worker's service-role client.
webhook_logs	Select where the form or the parent submission's form is owned by auth.uid().

The submission pipeline

This is the core of the engine: what happens between a visitor pressing Send and the owner's inbox. Order matters — cheap rejections happen before expensive work, and nothing is written to storage until the submission has passed every gate.

Step by step

1. Route match. POST /f/:formId loads the form by UUID. POST /email/:address is the zero-config path: it finds or creates a form for that address and, on first contact, emails a verification link and stops.
2. Body parse. form-body.ts reads multipart/form-data or x-www-form-urlencoded into a flat field map plus a list of File objects. Repeated field names are joined with commas so checkbox groups survive.
3. Special field parse. special-fields.ts extracts _next, _blacklist, _captcha, _webhook, _autoresponse, _subject, _cc, _replyto, _template, _format, and the honeypot, and records which of them were present for the audit trail.
4. Honeypot. If _honey or _honeypot has any value, the submission is dropped and the sender receives a completely normal success response. Bots get no signal.
5. Plan resolution. The owner's plan is loaded from profiles. Unclaimed zero-config forms fall back to DEFAULT_PLAN, which is free unless overridden for local testing.
6. Upload gate. Attachments on a free-plan form return a friendly upgrade message. Otherwise the total size of all files is checked against the 10MB cap before any R2 write is attempted.
7. Validation rules. Each configured field is checked for required and against its regex. Failures return a 400 with a structured error list, or an HTML error page for browser posts.
8. Captcha. If the form requires hCaptcha and the submission did not send _captcha=false, the token is verified server-side against hcaptcha.com/siteverify. A failure is rejected outright — this is the one spam signal that does not get stored.
9. Rate limit. A rolling one-hour counter in KV is incremented per form and per IP. Exceeding the form's limit returns 429 with a Retry-After header.
10. Monthly quota. Free plans are capped at 50 submissions per calendar month across all their forms; exceeding it returns a clear upgrade message rather than a silent drop.
11. Spam scoring. Per-submission _blacklist phrases are merged with the form's saved list, capped at 20 total, and matched case-insensitively against the visible field text. Link-count and keyword heuristics also run. Any hit sets is_spam.
12. File storage. Files are written to R2 at {formId}/{submissionId}/{filename} with sanitised, de-duplicated names.
13. Database insert. The submission is stored with its payload, file keys, spam flag, and meta audit trail. If the insert fails, the R2 objects just written are deleted so nothing is orphaned.
14. Owner notification. Unless the submission is spam, Resend sends the notification with the resolved subject, CC list, Reply-To, and template, including download links for any attachments.
15. Autoreply. If enabled, the submitter's address is detected from email, e-mail, or _replyto and a templated confirmation is sent. _autoresponse overrides the saved template for this submission.
16. Webhook fan-out. All configured destinations plus any one-off _webhook fire in parallel. Every attempt is logged; failures are queued for retry.

17. Response. JSON when `_format=json` or `Accept: application/json` was sent; otherwise a 302 to `_next`, then the form's `redirect_url`, then a built-in thank-you page.

The ordering rule

Rejections that cost nothing (honeypot, plan gate, validation) come before rejections that cost a network call (captcha) which come before anything that costs storage (R2, Postgres) which comes before anything that costs a third party (Resend, webhooks).

Response shapes

Status	Meaning
200 HTML	Thank-you page. Free plans get a small 'Powered by FormFlux' line; Pro does not.
302	Redirect to <code>_next</code> , else the form's <code>redirect_url</code> . Only http and https targets are honoured.
200 JSON	<code>{ "success": true, "submissionId": "..." }</code> — AJAX mode.
400	Validation errors as a field/message list, a captcha failure, or an oversized upload.
403	The target inbox has not been verified yet, so delivery is withheld.
429	Rate limit exceeded (with <code>Retry-After</code>) or the free monthly quota is used up.

SECTION 05

Special fields reference

Hidden inputs that change behaviour for a single submission. All of them are stripped from the payload before it is stored or emailed, so they never leak into the owner's inbox or the dashboard table. Every field below matches FormSubmit.co's documented behaviour.

Field	Behaviour
_replyto	Sets the Reply-To header. When absent, FormFlux auto-detects the submitter's address from an email or e-mail field, so replying always reaches the sender.
_next	Redirect target for this submission only, overriding the form's saved <code>redirect_url</code> . Non-http(s) values are ignored and logged rather than followed.
_subject	Notification email subject, overriding the form's <code>default_subject</code> .
_cc	Comma-separated additional recipients, overriding <code>default_cc</code> .
_blacklist	Comma-separated phrases. A case-insensitive match anywhere in the submitted text silently marks the submission as spam. Capped at 20 phrases.
_captcha	The literal string "false" skips hCaptcha for this submission even when the form requires it.
_honey	Honeypot. Any value means a bot filled a hidden field, so the submission is dropped silently. <code>_honeypot</code> is accepted as an alias.
_autoresponse	Autoreply body for this submission, overriding the saved template. Supports <code>{{field}}</code> placeholders.
_template	Email layout: "table" (default) or "plain". "basic" and "text" are accepted as aliases of plain for FormSubmit compatibility.
_webhook	Extra one-off webhook destination, posted with FormSubmit's exact <code>{ "form_data": { ... } }</code> body so copied examples work unchanged.
_format	Set to "json" for an AJAX-style JSON response instead of a redirect.

Override precedence

For every setting that exists in both places, the per-submission hidden input wins over the stored form setting. This is deliberate: the dashboard holds the sensible default, and a single page can deviate without a configuration change.

```
subject    : _subject      -> forms.default_subject  -> 'New FormFlux submission'
cc         : _cc           -> forms.default_cc       -> none
reply-to   : _replyto     -> detected email field  -> none
redirect   : _next        -> forms.redirect_url     -> built-in thank-you page
autoreply  : _autoresponse -> forms.autoreply_template
captcha    : _captcha=false -> forms.captcha_enabled
blacklist  : _blacklist + forms.blacklist_phrases (merged, capped at 20)
```

Why the blacklist is capped at 20

Matching every phrase against every field is $O(\text{phrases} \times \text{fields})$. Twenty is far more than any hand-written blacklist needs, and the cap stops a hostile submission from turning one request into thousands of substring scans.

SECTION 06

Spam protection, limits, and delivery

Four layers of spam defence

Layer	Behaviour
Honeypot	A hidden field bots fill and humans never see. Filled means dropped silently, with a normal success response so the bot learns nothing.
hCaptcha	Optional per form. Verified server-side. A failed token is the only spam signal that is rejected outright rather than stored.
Blocklist	Per-form phrases plus per-submission _blacklist. Matches are stored with is_spam = true and skip email and webhooks.
Heuristics	Excessive link counts and known spam keywords across the visible field text. Same soft outcome as the blocklist.

Soft spam handling is a feature, not a shortcut

Everything except a failed captcha is stored rather than discarded. The owner sees flagged submissions in the dashboard, can read exactly which rule fired from the meta.spamReasons list, and can mark a false positive as legitimate and resend the notification email.

Rate limiting

A rolling one-hour window in Cloudflare KV, counted independently per form and per IP. The limit is the form's rate_limit_per_hour, default 100. Exceeding it returns 429 with a Retry-After header telling the client exactly how long to wait.

File uploads

- 10MB across all files in one submission — a total cap, not per file, matching FormSubmit.
- Maximum 10 files per submission.
- Filenames are sanitised to a safe character set and de-duplicated with a short suffix when they collide.
- Stored in R2 under {formId}/{submissionId}/{filename} and served back through GET /files/... with a Content-Disposition attachment header.
- Download links appear in the notification email, the webhook payload, the submission drawer, and the archive view.

Webhooks

All destinations fire in parallel on every accepted submission. Each format has its own payload shape so the receiving system needs no adapter.

Kind	Payload
generic	FormFlux JSON: event, formId, formName, submissionId, createdAt, data, and files with signed-style download URLs.
slack	Slack Block Kit: a header, a submission reference, field sections, and a files section with links.
discord	A rich embed with the form name as the title and each field as an inline embed field.
formsubmit	FormSubmit-compatible { "form_data": { ... } }, used for the per-submission _webhook field.

RETRY BEHAVIOUR

A non-2xx response or a network failure enqueues a retry on Cloudflare Queues with exponential backoff — 60s, 120s, 240s, then 480s — for up to five total attempts. Every attempt writes a `webhook_logs` row with its status code, attempt number, and a 300-character response snippet, which is exactly what the dashboard's delivery log renders.

Autoreply and retention

- Autoreply detects the submitter from email, e-mail, or `_replyto` and renders `{{field}}` placeholders from the submitted data. The dashboard shows a live preview of the rendered template.
- A cron trigger at 03:00 UTC deletes submissions older than each form's `retention_days`, including their R2 objects. Setting 0 keeps submissions forever.

SECTION 07

The owner dashboard

A Next.js 14 App Router application that serves both the public marketing pages and the authenticated dashboard, sharing one design system: a warm terracotta accent reserved strictly for primary actions, stone neutrals, and a dark mode toggle in the header.

Routes

Route	Purpose
/	Landing page. Hero, a working demo form that posts to the Worker's live demo endpoint, a feature grid, a three-step how-it-works, and a footer.
/docs	Copy-paste snippets for zero-config, dashboard, and AJAX modes, plus a reference entry for every special field.
/pricing	Free versus Pro comparison with a Stripe Checkout stub behind the upgrade button.
/login	Supabase magic-link sign-in on a centred card, with inline email validation.
/forms	Grid of the owner's forms: verification badge, this month's submission count, spam count, and a copy-endpoint button. New accounts get a guided empty state.
/forms/[id]	Tabbed detail view: Submissions, Settings, Webhooks, Analytics.
/archive	Search across the owner's entire submission history, with a per-form filter and attachment links. No call limit.
/settings	Plan, usage against the current limits, and an explanation of how UUID endpoints keep the delivery address out of page source.

The four tabs

SUBMISSIONS

- Paginated, searchable, and filterable by all / not spam / spam.
- Row click opens a detail drawer with the full payload, attachment downloads, captcha outcome, which special fields the submission used, and the exact spam reasons if it was flagged.
- Mark a false positive as not spam, resend the owner notification, delete a submission, or export the whole history as CSV.
- Becomes a card layout on mobile instead of scrolling horizontally.

SETTINGS

- Every form column is editable here: target email, redirect, default subject and CC, webhook destinations, Slack and Discord URLs, captcha, blocklist phrases with a live count, autoreply with a live preview, validation rules, retention, and rate limit.
- The validation rule builder includes a per-rule 'test a sample value' tool that shows pass or fail as you type.
- A read-only 'copy this HTML' panel stays in sync with the current settings, so the snippet always matches the saved configuration.

- A sticky save bar reports unsaved changes, and navigating away with unsaved edits triggers a browser warning.

WEBHOOKS

Lists form-level destinations plus any per-submission _webhook URLs seen historically, each with a 'send test payload' button and immediate toast feedback, above a delivery log showing status code, attempt number, response snippet, and timestamp.

ANALYTICS

A 30-day submissions-per-day area chart in the accent colour with spam overlaid as a dashed line, plus spam-blocked count, webhook delivery success rate, and captcha pass rate.

Interaction standards applied throughout

- Every destructive action goes through a confirmation dialog.
- Every table shows loading skeletons rather than a blank flash.
- Every settings field validates inline with a red border and a message under the field.
- Every action reports through a toast — no browser alert() anywhere.
- Empty states always explain the next step instead of showing an empty table.

SECTION 08

Plans and feature gating

Gating is enforced in both the Worker and the dashboard from the same source of truth: `profiles.plan`. The dashboard shows a friendly upgrade prompt instead of hiding a feature, and the Worker returns an explicit `upgradeRequired` response rather than silently ignoring the request.

Capability	Free	Pro (\$7/mo)
Forms	1	Unlimited
Submissions / month	50	Unlimited
Email delivery	Yes	Yes
Spam protection & captcha	Yes	Yes
All special fields	Yes	Yes
Archive & CSV export	Yes	Yes
File uploads	No	Yes
Webhooks / Slack / Discord	No	Yes
Autoreply	No	Yes
Analytics	No	Yes
Custom domain	No	Yes
FormFlux branding	Shown	Removed

Deliberate difference from FormSubmit

FormSubmit limits reading your own submission archive to five API calls per day. FormFlux has no such limit on any plan, because the data lives in the owner's own Supabase project and is read directly under Row Level Security.

Build log — what was created

The project was built in five phases. Each phase extended the previous one rather than replacing it.

Phase 1 — Core submission engine

- Supabase schema with profiles, forms, and submissions under RLS.
- Hono app on Cloudflare Workers with POST `/f/:formId` and POST `/email/:address`.
- Multipart and urlencoded body parsing, honeypot, validation rules.
- Resend notification email with `_subject`, `_cc`, `_replyto`, and `_template` support.
- Inbox verification flow at GET `/verify/:token`, JSON versus redirect response handling.

Phase 2 — Spam protection and rate limiting

- Server-side hCaptcha verification gated on the form's `captcha_enabled` flag.
- Rolling one-hour KV rate limiting per IP and per form, returning 429 with Retry-After.
- Heuristic spam detection storing matches with `is_spam = true` instead of discarding them.

Phase 3 — Files, webhooks, autoreply, retention

- R2 uploads with sanitised keys and a GET `/files/...` download route.
- Webhook fan-out in generic, Slack, and Discord formats, logged to a new `webhook_logs` table.
- Cloudflare Queues retry with exponential backoff up to five attempts, plus a manual test-webhook endpoint.
- Autoreply with `{{field}}` placeholders and a daily retention cron that also deletes R2 objects.
- A local demo UI at the Worker root for inspecting every page state in a browser.

Phase 4 — Full FormSubmit parity

- New special-fields module implementing `_next`, `_blacklist`, `_captcha`, `_webhook`, and `_autoresponse`, plus `_honey` as a honeypot alias.
- 10MB total upload cap across all files in a submission, enforced before any R2 write.
- Reply-To auto-detection and `_template=basic` accepted for FormSubmit compatibility.
- Plan gating in the Worker, a submission meta audit trail, and authenticated dashboard endpoints (`/api/me`, `resend`, `spam toggle`).
- Migration 002 adding `default_subject`, `default_cc`, `blacklist_phrases`, `submissions.meta`, the generated `search_text` column, `webhook_logs.form_id` and `response_snippet`, and the new-user profile trigger.

Phase 5 — Dashboard and public site

- Next.js 14 dashboard with Tailwind, Radix-based shadcn-style primitives, dark mode, and toast feedback.
- Magic-link auth with middleware-guarded routes and a collapsible sidebar shell.
- Forms grid, four-tab form detail view, and the unlimited submission archive.
- Public landing page with a live demo form, documentation page, pricing page with a Stripe stub, 404, and an app-shell loading state.
- A new `/demo/submit` endpoint on the Worker that runs the real honeypot and spam checks but stores nothing, so the landing page demo works with zero configuration.

Verification performed

- Worker typechecks clean with `tsc --noEmit`.
- Dashboard typechecks and produces a successful production build (13 routes).
- Special fields exercised against the running Worker: `_honey` drops silently, `_next` redirects, `_format=json` returns JSON, and `_blacklist` flags spam while stripping itself from the stored payload.
- All public dashboard pages rendered and screenshotted in a real browser with no console errors; the landing page demo form completed a real submission.

Known gap in verification

The authenticated dashboard pages are typechecked and built but were not exercised against live data, because that requires a real Supabase project. Add real credentials to `dashboard/.env.local` and run the migration to complete this.

SECTION 10

Configuration and deployment

Worker environment

Variable	Purpose
SUPABASE_URL	Supabase project URL.
SUPABASE_SERVICE_KEY	Service-role key. Worker secret only — never exposed to a browser.
RESEND_API_KEY	Resend key for notification, verification, and autoreply email.
RESEND_FROM_EMAIL	Verified sender, e.g. FormFlux <forms@yourdomain.com>.
HCAPTCHA_SECRET	Required only when a form enables captcha.
PUBLIC_BASE_URL	Public Worker origin, used to build file download links.
DASHBOARD_URL	Public dashboard origin, used by upgrade prompts.
DEFAULT_PLAN	Plan for owner-less zero-config forms. free by default.

Dashboard environment

Variable	Purpose
NEXT_PUBLIC_SUPABASE_URL	Supabase project URL.
NEXT_PUBLIC_SUPABASE_ANON_KEY	Anon key. Safe in the browser; all access is filtered by RLS.
NEXT_PUBLIC_WORKER_URL	Worker origin the browser should call.
NEXT_PUBLIC_SITE_URL	Dashboard origin, used for magic-link redirects.
STRIPE_SECRET_KEY	Optional. Enables real Pro checkout; stubbed until set.
NEXT_PUBLIC_STRIPE_PRICE_ID	Optional. Stripe price for the Pro subscription.

Cloudflare bindings

```
npx wrangler kv namespace create RATE_LIMIT_KV
npx wrangler r2 bucket create formflux-uploads
npx wrangler queues create webhook-retries
# cron trigger: 0 3 * * * (daily retention sweep, already in wrangler.toml)
```

Deploy

```
# Worker
cd worker
npx wrangler secret put SUPABASE_SERVICE_KEY
npx wrangler secret put RESEND_API_KEY
npx wrangler deploy

# Dashboard (Cloudflare Pages)
cd dashboard
npm run build      # then connect the repo in Pages, or use @cloudflare/next-on-pages
```

Outstanding before real public traffic

1. Run `supabase/migrations/002_dashboard.sql` (or `schema.sql` on a fresh project).
2. Verify a sending domain in Resend and update `RESEND_FROM_EMAIL` — unverified accounts can only email the account owner's own address.
3. Create the production KV namespace, R2 bucket, and Queue, and paste the KV ids into `wrangler.toml`.
4. Point DNS at the Worker and the Pages project, then set `PUBLIC_BASE_URL` and `DASHBOARD_URL`.
5. Add the dashboard's `/auth/callback` URL to Supabase's allowed redirect list.
6. Add real hCaptcha keys if captcha will be used.
7. Add Stripe keys and finish the checkout route if paid upgrades are wanted.

SECTION 11

Workflow flowchart

End-to-end path of a single form submission through the FormFlux engine.

Outside the request path

ASYNC AFTER RESPONSE

Failed webhooks are queued and retried with exponential backoff — 60s, 120s, 240s, 480s — for five attempts total. Every attempt is logged.

DAILY AT 03:00 UTC

The retention cron deletes submissions past each form's retention_days and removes their R2 objects.

ALWAYS VISIBLE

The owner sees every submission in the dashboard, spam included, with the exact rules that fired.

NEVER A DEAD END

Each rejection returns a specific status and a readable reason — JSON for AJAX callers, an HTML page for browser posts.

